ParaDEVS Control

Specification: the paratemporal engine as a planner for RL action selection, value iteration and policy iteration (phases 4a–4c)

ParaDEVS Library workspace, Binghamton

October 16, 2026

How to read this document. Section 1 is a one-page summary. Every later section opens with a green "In short" box, so you can skim by reading only those boxes. Terms are defined in the glossary (Appendix A). Background is in the **RL Plan** and the **Build Report**.

ParaDEVS documents. The entry point is **Start Here**, a reading guide that links every document.

- **Build Report**: what has been built, how it was verified, what is next
- **How-To Guide**: setup, tutorials, API reference, troubleshooting
- **Architecture Guide**: how the library works and how to read results
- **RL Plan**: reinforcement-learning plan, analysis and results
- **Control Spec** (this document): action selection: Para.decide, graph-mode planning, value and policy iteration
- **Hybrid Study**: which draws to enumerate or sample, and how cost grows with lookahead
- **Fast-Copy Proposal**: why the engine replays steps, where time goes, how to speed it up

Contents

1 Summary 3

2 Definitions 4

3 Graph mode 5

4 Guarantees 6

5 API 8

6 How it plugs into value iteration, policy iteration and acting 9

7 Verification (phase 4c) 10

8 Phase plan 11

9 Open questions **12**

A Glossary **13**

1 Summary

Goal	Use the paratemporal engine to choose actions: at a real state, look n decisions ahead, compute the value $e(a)$ of each available action, and act greedily or ε-greedily. The same machinery runs value iteration and policy iteration.
What a model does	Calls `Para.decide(label, obs, options)` for each agent decision (a "do nothing" option is allowed) and `Para.choose` for environment noise, as before. The model does not change between planning, policy evaluation and plain MS4 simulation.
Key idea	**Graph mode.** The forward engine sums weights, which suits expectations but cannot take a max. The planner instead expands a graph to depth n (merging equal states at equal decision depth), records edges with probability, reward and elapsed time, and then runs a backward pass: weighted average at chance nodes, max (control) or policy average (evaluation) at decision nodes, leaf values from a batched hook.
Time	One step is one decision. DEVS events without a decision are not steps. Discounting is by simulation time, γ^τ (a semi-MDP); with one time unit per decision this is the usual γ^k.
Guarantees	Exact with exhaustive chance nodes, for the max and for the policy average. With sampled chance nodes the policy-weighted value stays unbiased; the sampled max is biased upward (measured; mitigations given).
Verified	**48 new checks pass** (all 147 library checks pass). Depth-n planning equals n steps of value iteration at all 64 FrozenLake states to 10^{-16}; engine-driven policy iteration and value iteration reach V^* to 3×10^{-12}; the merged graph needs 99× fewer simulation runs than the tree at depth 4.
Demo	A FrozenLake agent acting by 1-step lookahead with exact V^* leaves reaches the goal in 84–88% of episodes (exact 86.3%, the optimal policy's rate); with $V = 0$ leaves it reaches 0.2% at depth 1, 7.6% at depth 8, 57% at depth 14.
Next	4d learning comparisons (Python, tabular), 4e the Python wrapper, 4f a DEVS-native problem with random event times (Section 8).

Decision needed
Six questions are listed in Section 9. The most important: (1) whether the safe default time key (absolute times, merges less) is right, with the relative key as a declared opt-in; (2) the reward-timing convention for DEVS-native problems; (3) whether the meeting note's k-branch estimate $e(a)$ should average over the policy inside the lookahead (unbiased) or maximize (biased upward).

2 Definitions

In short
A *decision* is one call to `Para.decide`; it is one RL step. Lookahead depth counts decisions, not DEVS events. Rewards are read from the model's outputs and discounted by elapsed simulation time. What `decide` does depends on who runs the model: it samples a policy in plain MS4, becomes a policy draw in the forward engine, and becomes a decision node in the planner.

Decision epoch and step. A *decision epoch* is the moment a model calls `Para.decide(label, obs, options)` inside a transition. `obs` is what the agent observes (the input of the policy and of the leaf-value hook); `options` is the list of available actions, possibly including `Para.NOOP`. Each call is one step, even when only one option exists and even when the option is `NOOP`. DEVS events between two decisions (for example, an environment event or a report) are not steps: they are part of the transition from one epoch to the next.

Discounting. A reward output at simulation time t counts γ^{t-t_0} from an epoch at time t_0. The next epoch, reached after τ time units, is discounted by γ^{τ}. This is the semi-MDP convention (discrete lump rewards, continuous time). With $\tau = 1$ per decision it is the textbook MDP.

Lookahead to depth n. From a root state, the planner follows every option of every decision and every outcome (or a sample) of every draw, until n decisions have been made. The state at the next decision epoch is a *leaf*; its value comes from the *leaf-value hook* $V(obs)$. A branch that ends (the model becomes passive) before that is *terminal* and has value 0.

`Para.decide` by mode.

Who runs the model	**What `decide` does**	**Used for**
Plain MS4 simulation	samples an action from the installed policy `Para.setPolicy(p)`; the default is uniform over the options	running an agent; acting (a lookahead policy can be installed)
Forward engine (`ParaEngine`)	a draw from the installed policy's law; exhaustive, sampled or hybrid like any draw	policy evaluation with the full return distribution; n-step targets
Planner, *evaluate*	decision node; backward pass takes $\sum_a \pi(a \mid obs)\, Q(a)$	Q^π, V^π; policy iteration's evaluation step
Planner, *control*	decision node; backward pass takes $\max_a Q(a)$	value iteration; expectimax lookahead
Act (online)	plan from the real state with control (or evaluate), report $e(a) = Q(a)$ for every option, pick greedily or ε-greedily	online action selection

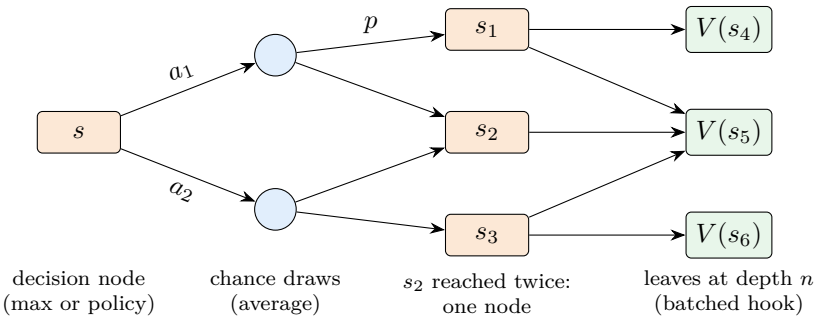
At the root decision the planner always reports $Q(a)$ for every option, whichever rule is used below it.

Targets. With V the learner's values at the leaves:

Target	Backward pass	Standard counterpart
Policy-weighted	policy average at every decision, leaf $V(s)$	n-step Expected SARSA, tree backup, V^π evaluation
Max at the leaves	policy average inside, leaf $\max_a Q(s,a)$ (a hook choice)	n-step Q-learning-style targets
Max inside the tree	max at every decision	n steps of value iteration, expectimax

3 Graph mode

In short
The planner turns the simulation into a layered graph. Nodes that reach the same model state at the same decision depth are merged, because their future value is the same. Rewards and elapsed times sit on the edges, not in the nodes. One backward sweep then computes every value. The expensive part, simulation, is done once; the cheap backward pass can be repeated with new leaf values.



Nodes.

- **Chance node**: a persistent model state (a completed departure) at decision depth d, after e events since the last decision. Its next event may make draws; each outcome (or sample) leads to another chance node or to a decision node.
- **Decision node**: a departure suspended at `decide`. Each option is resumed; the rest of the departure may make more draws. It is a leaf when $d = n$.
- **Edge**: (child, weight w, reward r, elapsed time Δt). The weight is the probability (exhaustive) or the sample factor $1/k$ (sampled); r is the discounted reward of the departure.

Merge keys. The key holds no accumulated reward: rewards are on the edges, so a node's value depends only on its state and on the remaining depth $n - d$. Under the Markov property and additive rewards this is exact for both the expectation and the max.

- Chance node: (model state key, d, e). Including e makes the graph layered and acyclic.

- Decision node: (pre-departure state key, event kind, values already drawn in this departure, d). It does not include e, so paths with different numbers of intermediate events still meet at their next decision.

Time in the key. The library's state key includes the simulator's last and next event times, which are absolute. Two settings (`PlanSpec.TimeKey`):

- **ABSOLUTE** (default): the node time and absolute event times are in the key. Sound for every model, but states reached at different times never merge.
- **RELATIVE** (declared opt-in): times are taken relative to the node time: the time to the next event $t_N - t$ for each component, and the elapsed time $t - t_L$ for components of a coupled model (they can receive inputs, and an external transition sees the elapsed time). A top-level atomic model receives no inputs under the planner, so its elapsed time cannot matter and is left out. This is sound for *time-homogeneous* models: transitions do not read the absolute clock except through model fields, and fields are always in the key (a field holding an absolute time just prevents merges). With random event times it matters: in the semi-MDP check the relative key needs 1,265 nodes and the absolute key 7,584 (Section 7).

A model that supplies its own key (`ParaStateKey`) can leave out `sigma`: its only role is the time advance, which the key already holds as the next event time.

Processing order. Nodes are expanded in the order (depth d; chance nodes before decision nodes; e). Every edge leads to a later node, so each node is expanded once, after all its arrivals, and a budget rule sees its full reach weight.

Backward pass. In reverse order,

$$\text{chance node:}\quad V(u) = \sum_{\text{edges } u \to c} w\,(r + \gamma^{\Delta t} V(c)),$$

$$\text{decision node:}\quad Q(u,a) = \sum_{\text{edges of } a} w\,(r + \gamma^{\Delta t} V(c)), \qquad V(u) = \textsc{rule}(Q(u,\cdot)),$$

with leaves $V(u) = \mathit{hook}(\mathit{obs}_u)$ and terminals $V(u) = 0$. The hook is called **once per backward pass** with every distinct leaf observation, so a Python learner is called once per plan.

4 Guarantees

In short
With every outcome enumerated, the planner's values are exactly those of dynamic programming. With sampled outcomes, the policy-weighted value is still correct on average. The sampled max is not: it is too optimistic on average, because the max picks up lucky samples. The fixes are to enumerate small draws, to sample more, or to choose and evaluate with independent samples.

Conditions throughout: finite option lists, finitely many draws per departure, finite-support laws for enumerated draws, a deterministic departure given the state and the supplied values (the library's standing requirement), and rewards that add along a path.

Proposition 1 (Exact with exhaustive chance nodes)**.** *With exhaustive selection, the backward pass with the max rule and $V = V_0$ at the leaves gives $V(root) = (T^n V_0)(s)$, where T is the Bellman optimality operator of the (semi-)MDP the model defines. With the policy rule it gives $(T_\pi^n V_0)(s)$. Root $Q(a)$ values are those of $T^{n-1}V_0$ (or $T_\pi^{n-1}V_0$).*

Why. Without merging, the graph is the expectimax tree, whose backward pass is T applied n times, by induction on the remaining depth. A merged node stands for several tree nodes whose futures are equal: equal keys mean equal continuation laws (safe merging, main_paratemporal Prop. "Safe merging"), the same remaining depth, and no dependence on the path's past reward, since rewards are on the edges. Their tree values are therefore equal, and the merged node's single value can be used for all. The edges keep their own w, r and Δt, so nothing path-dependent is lost. The max is applied to values that are each exact, so it is exact as well.

Proposition 2 (Unbiased policy-weighted target with sampled chance nodes)**.** *With any selection rule whose sample factors satisfy $E[\sum_i \alpha_i f(v_i)] = E_P f$ (exhaustive, `monteCarlo(k)`, `budget(B)`, hybrid by label), and with the policy rule, $E[\hat{V}(root)] = (T_\pi^n V_0)(s)$, also with merging.*

Why. The backward pass is linear in the leaf values and in the edge weights when the rule is the policy average. Conditionally on everything before a node is expanded, its draws are fresh, so the expected value of its estimate is its true value, by induction from the leaves. Merging only shares an expansion between parents; each parent's expectation is unchanged. This is the forward engine's argument (main_paratemporal, sampled case), applied backward.

Proposition 3 (Upward bias of the sampled max)**.** *With sampled chance nodes and the max rule, $E[\hat{V}] \geq V$, with strict inequality when the action ranking can change between samples.*

Why. Each $\hat{Q}(a)$ is unbiased, and $E[\max_a \hat{Q}(a)] \geq \max_a E[\hat{Q}(a)]$ (Jensen; the max is convex). Inside the tree the bias compounds, because each max sees already-optimistic children. This is the optimizer's curse, the same effect behind maximization bias in Q-learning.

Measured (FrozenLake, state 18, V^* leaves, so the exact value is $V^*(18) = 0.3755$; 200 seeds).

Chance nodes	**Mean**	**Bias**	**s.e.**
exhaustive, depth 3	0.375496	0 (exact)	0
`monteCarlo(1)`, depth 1	0.4119	+0.036	0.0026
`monteCarlo(3)`, depth 1	0.3919	+0.016	0.0020
`monteCarlo(3)`, depth 3	0.4103	+0.035	0.0017
double estimation at the root, $k = 1$, depth 1	0.3169	−0.059	0.0106

Mitigations.

1. **Enumerate small draws** (hybrid selection: `exhaustive` on labels with few outcomes, sampling on the rest). Where every draw is enumerated, the bias is zero. This is the engine's main advantage over plain sampling-based planners (Build Report and RL Plan: exhaustive expansion with merging wherever outcome sets are small).
2. **Sample more**: the bias shrinks as k or B grows (+0.036 → +0.016 from $k = 1$ to $k = 3$ at depth 1).

3. **Double estimation**: select the action with one sampled plan and evaluate it with an independent one (double Q-learning). It removes the upward bias (it may err downward, as measured). At the root this is two plans; inside the tree it needs two independent sample sets per node (future work).
4. **Use the policy rule for targets**: for learning targets, the policy-weighted target is unbiased; keep the max for action selection, where only the ranking matters.

5 API

In short
A model calls `Para.decide`. A caller builds a `PlanSpec`, expands a `Plan` from one or many roots, and runs `backup` with a rule and leaf values. All inputs and outputs are lists and arrays, so the Python side can call it in batches. A `LookaheadPolicy` lets an agent act by planning inside a plain MS4 run.

Model side (`examples/rl/MdpAgent.java`, abridged).

```
if (phase == Phase.DECIDE) {
    Object a = Para.decide("action", s, options());     // one RL step
    int s2 = a == Para.NOOP ? s
           : Para.choose("transition", mdp.transitionLaw(s, (Integer) a));
    reward = a == Para.NOOP ? 0 : mdp.rewardOnArrival(s2);
    s = s2; phase = Phase.REPORT; sigma = 0;           // report at the same time
} else if (phase == Phase.REPORT) {                    // an event, not a step
    reward = 0;
    if (mdp.isTerminal(s)) { phase = Phase.DONE; sigma = INFINITY; }
    else { phase = Phase.DECIDE; sigma = duration(); } // 1, or a "duration" draw
}
```

Planning side (package `paradevs.engine`).

```
PlanSpec spec = PlanSpec.depth(n)              // n decisions ahead
    .gamma(0.99)                               // per unit of simulation time
    .selection(SelectionRule.exhaustive())     // or monteCarlo(k), budget(B), hybrid
    .timeKey(PlanSpec.TimeKey.RELATIVE);       // opt-in: time-homogeneous models

Plan plan = Planner.expandFresh(spec, roots);  // many roots, one shared graph
List<Object> obs = plan.leafObservations();    // distinct leaf observations
Plan.Values v = plan.backup(DecisionRule.max(), leafValues);  // double[], as obs
// or: plan.backup(DecisionRule.policy(pi), LeafValues.table(V))

double value = v.value(i);                     // V(root i)
double[] q   = v.q(i);                         // e(a) = Q(root decision, a)
List<?> acts = v.actions(i);                   // the options, in model order
double[] p   = v.epsilonGreedy(i, 0.1, 1e-12); // epsilon-greedy, ties shared
Plan.Stats w = plan.stats();                   // departures, nodes, merged, ms
```

Other pieces: `Planner.Root.snapshot(model, now)` plans from a model in the middle of a plain MS4 run; `PlanSpec.prune(pi)` expands only the actions a policy can take (below the root); `PlanSpec.merge(false)` gives the plain tree, for comparison. `Para.Policy` is `double[] probabilities(label, obs, options)`; `LeafValues` is `double[] values(List<Object> obs)` (a batch), with `LeafValues.of(v)` for a single-leaf function, `table(v)` and `zero()`.

Acting under plain MS4. Install a lookahead policy; every `decide` then plans from the current observation.

```
Planner.LookaheadPolicy act = Planner.lookahead(
    PlanSpec.depth(1).gamma(0.99),
    obs -> new MdpAgent("frozenLake8x8", (Integer) obs),  // first decision at obs
    DecisionRule.max(), LeafValues.table(vStar),
    0.0,                                                  // epsilon
    true);                                                // cache by observation
act.prepare(allStates);      // optional: one batched plan for many states
Para.setPolicy(act);         // then run the model in MS4 as usual
```

Without an installed policy, `decide` under plain MS4 samples uniformly over the options (documented default). The cache is valid with exhaustive selection and fixed leaf values, where a plan never changes.

From Python (sketch; the wrapper is being built separately, phase 4e).

```
plan = Planner.expandFresh(spec, roots)        # roots built on the Java side
obs  = list(plan.leafObservations())
leaf = JArray(JDouble)(learner.V(obs))         # one learner call per plan
vals = plan.backup(DecisionRule.max(), leaf)
q    = np.array(vals.q(0))                     # e(a) at the real state
```

6 How it plugs into value iteration, policy iteration and acting

In short
Expand once, back up many times. Value iteration repeats the max backup with the previous values at the leaves; policy iteration alternates policy backups (evaluation) with a greedy choice from the root Q values (improvement); acting plans from the real state and picks from $e(a)$.

- **Value iteration.** Expand a depth-n plan from every state (one shared graph). Iterate $V_{k+1}(s) =$ `backup(max, `V_k`).value(s)`; each sweep applies T^n. No simulation is repeated.
- **Policy iteration.** Evaluation: iterate the policy backup to a fixed point (or use the forward engine for the whole return distribution). Improvement: $\pi'(s) = \arg\max_a$ `q(s)[a]` with the leaves at V^π, keeping the current action on ties.
- **Online action selection** (the priority). At the real state, plan to depth n with the learner's V at the leaves, read $e(a) = Q(a)$ for every option, and act greedily or ε-greedily. The meeting note's "play each action forward k times" is `selection(monteCarlo(k))` with `merge(false)`; merging and enumeration of small draws are the engine's additions. Learners remain standard:

the engine supplies $e(a)$, the learner updates V or Q from real transitions (or from engine targets).

- **n-step targets for TD learning.** The policy backup at a real state gives an unbiased n-step expected target, generalizing the existing forward-engine targets (`NStepTargets`) to models with agent decisions.

7 Verification (phase 4c)

In short
The planner was checked against exact dynamic programming on the 19-state random walk and slippery FrozenLake 8×8. Every exact check matches to 10^{-12} or better, the merged graph is far cheaper than the tree, the sampled estimators behave as Section 4 predicts, and the online demo reaches the optimal success rate. Test: `bash scripts/build_library.sh control` (about one minute).

Check	Result	Work
Depth n, $V = 0$, max: V and $Q = n$ steps of VI at every state; random walk $n = 1, 5, 12$	error 0	137–1,204
Same, FrozenLake $\gamma = 0.99$, $n = 1, 3, 10$, all 64 states	error $\leq 1.1 \times 10^{-16}$	1,076–9,707
Same, with a no-op action ($n = 4$), against DP with a "stay" action	error $\leq 1.1 \times 10^{-16}$	
Semi-MDP: step durations $\{0.5, 1, 2\}$ w.p. $\{\frac{1}{4}, \frac{1}{2}, \frac{1}{4}\}$, $\gamma = 0.9$, $n = 4$, both time keys	error 2.8×10^{-17}	5,013 (rel.), 33,477 (abs.)
Uniform π, depth 2, V^π leaves: V^π, Q^π, greedy improvement	1.8×10^{-13}, 9.1×10^{-14}, 53/53 states agree	
Policy iteration on one depth-1 graph, from "always Left"	11 improvements, 6,944 backups; $\lvert V - V^* \rvert \leq 3.0 \times 10^{-12}$, optimal policy	1,076
Value iteration on the same graph	1,027 backups; $\lvert V - V^* \rvert \leq 3.0 \times 10^{-12}$	1,076
`decide` under the forward engine (policy draws) vs the planner, $n = 3$	equal to 10^{-15}; both $= V^\pi(18)$	
Plan from a model in the middle of a plain MS4 run (`Root.snapshot`) vs a fresh model	Q equal (error 0)	
`decide` under plain MS4 with the default policy	the episode runs to its end	
Sampled, policy rule: `monteCarlo(3)`, `budget(12)`, hybrid (durations sampled)	unbiased, within 0.7, 0.9, 1.6 s.e.	237, 861
Sampled, max rule	upward bias measured (Section 4)	

Work counts departures (simulation runs of one event, including replays).

Graph versus tree (FrozenLake, state 18, the same value to 10^{-12}).

Depth	Graph departures	Graph nodes	Tree departures	Ratio
2	89	37	399	4.5×
4	478	143	47,379	99×

The tree grows like 12^n (4 actions, 3 slips); the graph grows linearly in n, bounded by the number of states per depth. A depth-10 plan from all 64 states takes 9,707 departures, about one second.

Online action selection demo (FrozenLake 8×8, plain MS4 simulation, the agent calls `decide` and a `LookaheadPolicy` plans; greedy, ties shared; success = goal within 200 decisions; 500 episodes; the exact rate of each induced policy by DP).

Leaves, depth	Simulated	Exact	$e(a)$ at the start (L, D, R, U)	Plan departures
V^*, $n = 1$	0.840	**0.8630**	0.4095, 0.4137, 0.4137, 0.4146	1,076
V^*, $n = 3$	0.884	**0.8630**	same	2,994
$V = 0$, $n = 1$	0.002	0.0024	0, 0, 0, 0	1,076
$V = 0$, $n = 4$	0.016	0.0132	0, 0, 0, 0	3,953
$V = 0$, $n = 8$	0.076	0.0932	0, 0, 0, 0	7,789
$V = 0$, $n = 14$	0.572	0.5709	0, 0, 0, 0	13,543
$V = 0$, $n = 30$	0.702	0.7191	0.0230, 0.0276, 0.0276, 0.0284	28,887

With exact leaves, one step of lookahead already gives the optimal policy (for $\gamma = 0.99$), whose success rate is 0.863. With $V = 0$ the agent sees nothing until the goal is within n decisions; the goal is 14 steps from the start, so small n is nearly random. This is the gap a learned V is meant to close (phase 4d). All plans for the 64 states were computed in one batched call and cached.

8 Phase plan

In short
4a–4c are done (this document, the engine and the exact checks). Next is the learning comparison in Python, the Python wrapper, and a problem where DEVS time matters.

Phase	Work	Done when
4a Spec	this document	reviewed by the user and advisor
4b Engine	`Para.decide`, `Planner`, `Plan`, `PlanSpec`, `LeafValues`, `DecisionRule`, `LookaheadPolicy`; `MdpAgent` model	done; all tests pass
4c Exact DP checks	`VerifyControl` (48 checks, Section 7)	done
4d Learning comparisons	Tabular learners in Python (Q-learning, n-step SARSA, Expected SARSA / tree backup) on the random walk and FrozenLake, against the same learners helped by the engine: (i) action selection by lookahead with the learned V at the leaves; (ii) engine n-step targets; (iii) k-branch $e(a)$ (sampled) vs exhaustive with merging. Measures: return and value error per real episode, and per unit of compute (departures), at equal compute; ε-greedy over $e(a)$	learning curves with confidence bands; a compute-normalized comparison
4e Python	JPype wrapper: `plan(states, n, rule, leaf_fn)` returning $e(a)$ arrays; a Gymnasium environment whose `step(action)` runs the MS4 model; one learner call per plan	Python reproduces the 4c numbers
4f DEVS-native problem	T4 of the RL Plan: admission control or an (s, S) inventory with random event times, several events between decisions, a coupled model, the relative time key	DP on the embedded chain matches; learning comparison repeated

Later: neural networks for V (the batched hook is designed for this); double estimation inside the tree; expectation-only merging for the forward engine (being added separately) used for policy-weighted targets.

9 Open questions

In short
Six choices were made to get a working first version. Each is safe, but the user or advisor should confirm or change them.

1. **Time key default.** ABSOLUTE is sound for every model but merges less; RELATIVE must be declared and is sound for time-homogeneous models. Keep this default?
2. **Reward timing.** `MdpAgent` reports the reward at the decision time, so the textbook MDP is recovered with $\tau = 1$. In a semi-MDP, rewards earned at the end of a sojourn would be discounted by γ^{τ}. Which convention for DEVS-native problems (lump sums when output, or reward rates)?
3. **The meeting note's $e(a)$.** Should the k-branch estimate average over the policy below the first action (unbiased) or maximize (biased upward, Section 4)? The API supports both.
4. **Leaf input.** The hook receives the `obs` passed to `decide`. That is right when `obs` is a Markov state; for partially observed DEVS models the hook may need the model state instead.
5. **Default policy under plain MS4.** Uniform over the options. Alternatives: the first option, or an error when no policy is installed.

6. **Merging across event counts.** Chance nodes include the number of events since the last decision, so the graph is acyclic; decision nodes do not. Merging chance nodes across event counts would need cycle handling. Is the current trade-off acceptable?

A Glossary

Decision (step)	One call to `Para.decide`; one RL step.
Decision epoch	The moment of a decision, with its observation and options.
No-op	The "do nothing" option `Para.NOOP`; choosing it is still a step.
Draw	One call to `Para.choose`: a random value with a known law (environment noise).
Departure	One run of one DEVS event from a saved state; the unit of work.
Chance node	A model state between decisions; its draws are averaged.
Decision node	A suspended `decide`; its options are maximized over or averaged by a policy.
Leaf	A decision node after n decisions; valued by the leaf-value hook.
Terminal	A branch whose model became passive; value 0.
Merge key	What two nodes must share to be joined: state, decision depth (and, for chance nodes, events since the last decision).
Time key	Whether times enter the key as absolute values or relative to the node time.
Backward pass (backup)	The sweep from leaves to roots that computes all values.
$e(a)$**,** $Q(s,a)$	The value of taking action a now and then following the rule.
Policy-weighted target	Values averaged over the policy at every decision; unbiased under sampling.
Value iteration	Repeating the max backup until values stop changing; gives V^*.
Policy iteration	Alternating evaluation of a policy and greedy improvement.
ε**-greedy**	Act on the best $e(a)$ with probability $1-\varepsilon$, uniformly otherwise.
Semi-MDP	An MDP in which steps take random time; discounting by γ^τ.
Selection rule	How draws are expanded: exhaustive, `monteCarlo(k)`, `budget(B)`, hybrid by label.
Optimizer's curse	The max of noisy estimates is too high on average.
Double estimation	Choosing with one estimate and evaluating with an independent one.
Expectimax	A lookahead tree with max at decisions and averages at chance nodes.
MDP homomorphism	A map to a smaller MDP that preserves transitions and rewards; exact merging is a special case.